Learning Goal-Directed Rolling: Spherical Robot Point-to-Point Control Through Reinforcement Learning

Junjie An, Runhua Zhang, Yifan Liu, *Member, IEEE*, Xiaoqing Guan, *Member, IEEE*, You Wang, *Member, IEEE*, Jie Hao, and Guang Li

***Abstract*—Point-to-point navigation is an important ability for spherical robots. Traditional methods usually use a planner and a tracker for short-range target control. However, this hierarchical method suffers from a mismatch issue. In this work, we propose an end-to-end controller based on reinforcement learning. The proposed approach is designed for waypoint tracking after a path has been planned. Taking proprioceptive information such as the robot's position and orientation as input, our controller directly outputs motor commands to control the spherical robot. To adapt to the unique characteristics of a spherical robot, we have designed various reward functions, a long history encoder, and curriculum learning. We demonstrate that our policy can execute point-to-point tasks with high efficiency stability and adaptability to uncertain environments, achieving a success rate of 88.87% in simulation. To transfer the policy trained in simulation to the real world, we developed a MC-CMA-ES method for system identification to accurately model the simulator's parameters. This process significantly narrows the gap between simulation and reality, enabling our policy to achieve high stability and efficiency in real-world scenarios.**

***Index Terms*—Point-to-Point control, spherical robots, system identification.**

I. Introduction

SPHERICAL robots constitute a novel and distinctive class of mobile robots, characterized by their fully enclosed spherical structure. Locomotion is achieved through rotation of the outer shell for forward and backward movement, while directional changes are controlled by shifting an internal pendulum [1], [2]. Compared to other robotic platforms, the closed-shell design of spherical robots offers enhanced protection against tipping and external impacts, thereby shielding internal electronics from damage. These advantages make spherical robots particularly well-suited for applications such as security inspection and search-and-rescue missions.

Despite the potential, spherical robots present considerable control challenges due to their inherently nonlinear and underactuated dynamics. Achieving accurate and reliable control remains a critical research issue.

Current navigation framework comprises two components: a trajectory planner and a trajectory tracker [3], [4]. Upon receiving a goal position, the planner computes an optimal trajectory based on the robot's kinematics, and the trajectory tracker employs dynamic control to follow the planned trajectory [5]. These traditional control strategies heavily rely on accurate dynamic models. However, the ideal dynamic model often fails on irregular terrains and when encountering disturbances. This hierarchical architecture also has limitations. Each component requires independent tuning, increasing error accumulation. When the error becomes excessive and a mismatch occurs between the two layers, the overall system stability is significantly reduced.

To address these challenges, we propose a reinforcement learning-based end-to-end control framework for spherical robots. This method simplifies the need for separate planning and tracking stages by directly mapping sensory inputs to low-level control commands via a neural network policy. End-to-end approaches have gained substantial attention in recent robotic control research and have demonstrated success in quadrupedal and humanoid systems [6], [7], [8], [9]. By learning a control policy through reinforcement learning, the robot can achieve rapid and stable point-to-point navigation without reliance on accurate dynamic models [10]. Reward function design plays a critical role in guiding the learning process to ensure both efficiency and smooth motion.

However, transferring policies trained in simulation to real-world deployment remains a major obstacle. Performance is often bad when using policies directly, due to the discrepancies in perception, actuation, and environmental dynamics between simulation and real world. To address this, we introduce a system identification method to narrow the sim2real gap.

In summary, our main contributions include:

- We propose a RL-based end-to-end approach to solve the point-to-point control problem for a spherical robot. In contrast to traditional hierarchical schemes, our method shows

Received 7 October 2025; accepted 18 January 2026. Date of publication 16 February 2026; date of current version 24 February 2026. This article was recommended for publication by Associate Editor S. C. Ryu and Editor C. Gosselin upon evaluation of the reviewers' comments. This work was supported in part by the State Key Laboratory of Industrial Control Technology, China, under Grant ICT2024A21 and in part by Rotunbot (Hangzhou) Technology Company, Ltd., fund. *(Corresponding author: You Wang.)*

Junjie An, Runhua Zhang, Xiaoqing Guan, You Wang, and Guang Li are with the State Key Laboratory of Industrial Control Technology, Institute of Cyber Systems and Control, Zhejiang University, Hangzhou 310027, China (e-mail: 12332047@zju.edu.cn; 22432061@zju.edu.cn; xiaoqing_guan@zju.edu.cn; king_wy@zju.edu.cn; guangli@zju.edu.cn).

Yifan Liu is with the State Key Laboratory of Industrial Control Technology, Institute of Cyber Systems and Control, Zhejiang University, Hangzhou 310027, China, and also with the School of Automation Science and Engineering, South China University of Technology, Guangzhou 510641, China (e-mail: yifanliu@scut.edu.cn).

Jie Hao is with The Luoteng Hangzhou Techonlogy Company, Ltd., Hangzhou 310018, China (e-mail: mac@rotunbot.com).

This article has supplementary downloadable material available at https://doi.org/10.1109/LRA.2026.3665076, provided by the authors.

Digital Object Identifier 10.1109/LRA.2026.3665076

advantages in adaptability to uncertain environments and overall coherence.
- To effectively address the point-to-point challenge on spherical robots, curriculum learning and an LH Encoder are introduced. The proposed method demonstrates significant improvements in stability, efficiency, and system simplicity.
- This work is also the first time to apply a sim-to-real reinforcement learning approach to a spherical robot. We apply a Multi-populations Cooperative Covariance Matrix Adaptation Evolution Strategy (MC-CMA-ES) approach to the parameters identification on our spherical robot. Our approach demonstrated a high success rate in real-world experiments.

II. Related Work

A. Control of Spherical Robots

Our current navigation framework for spherical robots adopts a hybrid approach, combining a trajectory planner based on Differential Flatness [11] with a trajectory tracker based on Model Predictive Control (MPC) [1], [12], [13], [14]. These model-based methods rely on the robot's kinematic or dynamic models to compute optimal control strategies [15]. However, such methods are heavily dependent on the accuracy of the underlying models, which limits their robustness in the face of disturbances and reduces their adaptability in novel or unstructured environments. In addition, their reliance on numerical solvers incurs substantial computational costs, making it difficult to achieve fast response times and efficient motion. Dwaracherla [16] uses PID to control the ball towards the goal. But this feedback control method performs poorly in terms of stability and efficiency.

In contrast, model-free reinforcement learning (RL) controllers offer improved robustness and performance optimization by leveraging randomized training environments and carefully designed reward functions [10]. This approach has gained widespread success in recent years, particularly in controlling complex robotic systems [17], [18], [19], [20].

Several studies have also demonstrated the effectiveness of RL in enabling point-to-point control of specialized mobile platforms such as quadruped robots and Autonomous Underwater Vehicles (AUVs) [21], [22], [23]. However, to date, reinforcement learning has not been widely applied to spherical robots. Given their highly nonlinear and underactuated nature, further research is needed to explore how RL-based methods can be adapted for reliable and efficient control in this domain.

B. Sim-to-Real Transfer

Policies trained in simulation often face substantial performance drops when deployed in real-world settings, a challenge commonly referred to as the sim-to-real gap. A number of strategies have been proposed to address this issue.

System identification is one of the most direct approaches. It involves collecting motion data from the physical robot to estimate its dynamic model, which can then be used for model-based reinforcement learning [24], [25]. While this method can yield accurate models for specific scenarios, it lacks generalizability and typically requires large amounts of experimental data, making it costly and time-intensive.

Domain randomization offers an alternative by introducing randomized variations in simulation parameters—such as friction, mass, and sensor noise—during training [21], [26]. This enhances policy robustness across a wide range of real-world conditions. However, excessive randomization may hinder learning convergence or degrade the final policy performance if the training distribution deviates too far from reality.

The third approach is real-world fine-tuning, which includes human-in-the-loop correction methods [27] and online policy adaptation in physical environments [28], [29]. While effective in many cases, real-world training of spherical robots poses safety risks due to their enclosed structure and dynamic behavior, making such approaches less straightforward to implement.

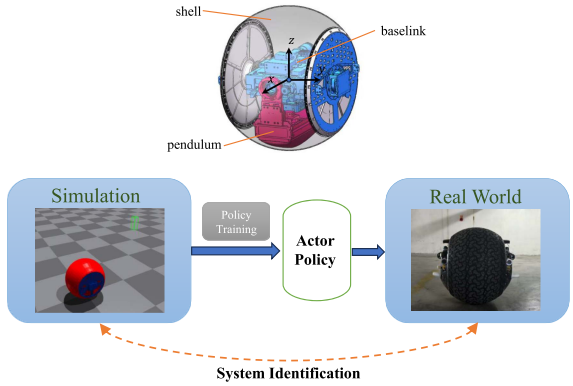

Fig. 1. Overview of point-to-point control. The upper figure is the mechanical design of the spherical robot.

III. Method

In this section, we introduce an end-to-end control method for spherical robots based on reinforcement learning. We train the policy of the spherical robot to autonomously navigate to the target point on the plane. The policy provides joint position or velocity commands for the two joints based on the state information that sensors can obtain.

Our approach consists of three steps: system identification, training in the simulator, and experimentation in the real robot. The framework is shown in Fig. 1.

A. Spherical Robot Hardware

The spherical robot is driven by a 2-DOF heavy pendulum [1], [30], [31], and the mechanical schematic of the robot can be seen in Fig. 1. Our spherical robot has a diameter of 0.8 m. The masses of the pendulum, shell, and center are approximately 80 kg, 10 kg, and 25 kg. A 2-DOF heavy pendulum driven by two motors is employed to drive the robot. The y-axis motor is employed to control the velocity, while the x-axis motor is employed to control the direction. The sensors used in this project include IMU, GPS, and wheel speedometer.

In these two joints, the first motor controls the forward rolling of the spherical robot, and its main control method is PID-based speed control. The second motor controls the swing of the heavy pendulum to control the direction of the robot, and its main control method is PID-based position control.

B. System Identification

Since the physical laws and robot models in the simulation environment are quite different from those in the real

environment, transferring the learning results to the real environment faces several challenges. In this section, we introduce a system identification (Sysid) approach to minimize the sim2real gap based on Covariance Matrix Adaptation Evolution Strategy(CMA-ES) [25]. We assume that π_{sim} is the policy given in the simulator. f_{real} represents the physical function of the real world, which is a "Closed Box" system. f_{sim}^{ϕ} represents the physical function of the simulation environment under the parameter ϕ_{sim}.

$$s_{t+1}^{real} = f_{real}(s_t^{real}, a_t),\ s_{t+1}^{sim} = f_{sim}^{\phi}(s_t^{sim}, a_t, \phi)$$

where ϕ_{sim} contains various parameters in the simulation, including friction coefficients, mass, inertia, stiffness and so on. Policy π_{sim} provides a series of action sequences $\{a_0, a_1, \ldots\}$for the system. From the initial state s_0, we let $\tau_{sim} = \{s_0^{sim}, s_1^{sim}, \ldots\}$ denote the trajectory generated by π_{sim} in the simulator and let $\tau_{real} = \{s_0^{real}, s_1^{real}, \ldots\}$ denote the trajectory generated by π_{sim} in the real world. The goal of the system identification problem is to solve the following optimization formula:

$$\phi^* = \arg\min_{\phi \in \Phi} \sum_{t=0}^{T} \|f_{real}(s_t^{real}, a_t) - f_{sim}^{\phi}(s_t^{sim}, a_t, \phi)\|^2 \tag{1}$$

where ϕ^* is the optimal system parameters, f_{real} and f_{sim} are all regarded as closed box functions. This distribution optimization problem can be solved with CMA-ES, which uses a normal distribution $N(m_k, \sigma_k^2 C_k)$ to search for optimal parameters. The process is iterated to update the mean μ_k, step size σ_k, and covariance matrix C_k of the normal distribution after each search. Our simulator Isaac Gym has the advantage of supporting massively parallel simulations and GPU-accelerated engine, so we can search parameters on thousands of spherical robot models. Moreover, we provide multiple different policies to cover as many operating conditions as possible. Policies generate different trajectories $\tau_{sim}^1, \tau_{sim}^2, \ldots, \tau_{sim}^n$ in the simulator and $\tau_{real}^1, \tau_{real}^2, \ldots, \tau_{real}^n$ in the real world. We formulate the goal of CMA-ES with average trajectories loss:

$$\mathcal{J}(\phi) = \frac{1}{N} \sum_{n=1}^{N} \|\tau_{sim}^n(\phi) - \tau_{real}^n\|^2, \phi \sim \Phi \tag{2}$$

Using CMA-ES in the simulation is computationally slow. Furthermore, due to the large number of parameters, it is prone to getting trapped in local optima. Therefore, we propose Multi-populations Cooperative CMA-ES (**MC-CMA-ES**), which expands the search range to avoid these local optima. We divide the entire population into multiple subpopulations to improve efficiency. Each subpopulation has different initial values and mutation seeds. The multi-population CMA-ES approach has been mentioned in several works [32], [33], [34]. We apply this method to the parameter identification for the spherical robot, and further improve it by incorporating an immigration strategy to enhance search efficiency. This approach also leverages the advantages of the IsaacGym environment. In each iteration, the optimal solution for each subpopulation and the global optimal solution are obtained. After a certain number of iterations, an immigration policy is implemented, where the top λ individuals of the best population $x_{i:\lambda}$immigrate to the worst population w.

Algorithm 1: MC-CMA-ES Based SysID.

Input: action sequences A, real world trajectory τ_{real}
Output: optimal parameters ϕ^*

Initialize MC-CMA-ES: ϕ_0, Φ_0, M(population size), N_p(num of races), K_i
for $k = 1 : K$ **do**
  for $n = 1 : N_p$ **do**
    for $i = 1 : M$ **do**
      Sample $\phi_i^n \sim \Phi_{k-1}^n = N(m_{k-1}, \sigma_{k-1}^2 C_{k-1})$;
      Run action A in simulation with ϕ_i and collect τ_{sim};
      Calculate $\mathcal{J}(\phi_i)$ with τ_{sim} and τ_{real} ;
    end
    $\phi_n^* \leftarrow \arg\min \mathcal{J}(\phi_i^n)$;
    $\Phi_k^n \leftarrow$ *updateCMAES*(ϕ_n^*, Φ_{k-1}^n) ;
  end
  $\phi^* \leftarrow \arg\min \mathcal{J}(\phi_n^*)$;
  if $k \mod K_i = 0$ **then**
    Find the worst performing parameters ϕ_w^*;
    $\Phi_k^w =$ *Immigrate*(ϕ^*, Φ_k^w) ;
  end
end
return optimized ϕ^*

The parameter of w is updated with $Immigrate(\phi^*, \Phi_k^w)$:

$$m_k^w = \omega_0 m_k^w + \sum_{i=1}^{\lambda} \omega_i x_{i:\lambda} \tag{3}$$

$$C_k^w = (1 - c_\lambda) C_k^w + c_\lambda \sum_{i=1}^{\lambda} \omega_i \left(\frac{x_{i:\lambda} - m_k^w}{\sigma_k}\right) \left(\frac{x_{i:\lambda} - m_k^w}{\sigma_k}\right)^T \tag{4}$$

The entire algorithm's workflow is summarized in Algorithm 1.

C. RL Goal and Observation

In the end-to-end navigation task, our goal is to give any point in the restricted areas, and the spherical robot will autonomously navigate to the target point. In this task, the motion area of the spherical robot is a square, $x \in [-5, 5], y \in [-5, 5]$. The robot's initial position is always in the center, and the initial orientation is randomly given. In each epoch, the target point p_o will be randomly assigned. A circle with a radius of 0.5 that cannot be selected is given to prevent the robot from starting too close to the goal. The task is considered successful when the distance to the goal is less than $d_s = 20$ cm and the velocity is less than $v_s = 0.1$ m/s

Observation: Theoretically, the dynamic equations of a spherical robot are related to its orientation, velocity, angular velocity, and joint torques [12]. The observations include the base pose, velocity, and joint information. This work uses the PPO algorithm for reinforcement learning, an asymmetric actor-critic algorithm. The critic network obtains observations consisting of accurate privileged information from the simulation, such as environmental friction, robot mass, and contact forces. The privileged observations are used to evaluate the quality of the

TABLE I
OBSERVATIONS FOR THE ACTOR AND CRITIC

Observations			Dim
o_t	$\mathbf{p}_c$	command, position of target point	2
	p_b	robot current position	2
	o_b	robot current orientation (Euler angle)	3
	v_b	robot base linear velocity	3
	ω_b	robot base angular velocity	3
	q_j	joint angle of second axis	1
	ω_j	joint velocity	2
	a_{t-1}	previous action	2
o_t^{pr}	f_e	environment friction	2
	F_c	contact force between ball and ground	3
	F_p	random push force	3

spherical robot's motion. On the other hand, the actor network represents the actual motion policy of the spherical robot. All observed quantities are described in Table I.

Noise: Noise is added to the observation because the sensor readings in the real environment have errors and delays. We use RMSE (Root Mean Squared Error) to quantitatively determine the noise level with data from the real spherical robot. In each set of trajectories τ_{sim} and τ_{real}, the noise of the k-th observation is

$$u_k = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(o_t^k - \hat{o}_t^k)^2}$$

where o_t^k is the observation in the real world and $\hat{o}_t^k$ is the observation in the simulator. We also use a random delay of 0-0.1 seconds.

To enhance the controller's robustness against uneven terrain and disturbances, a randomly uneven terrain and a random force is established in the simulation platform.

D. Rewards

In this section, we design several reward functions for the end-to-end task. Our reward functions include positive reward, negative penalty, and soft constraints. All the components are shown in Table II.

Positive rewards are the dominant factor for the spherical robot to reach the target point, divided into four parts. The term success encourages the spherical robot to reach the target point and stop. The term distance encourages minimizing the distance between the spherical robot and the target goal [23]. The term approaching target encourages robots moving towards the goal preventing the spherical robot from rolling aimlessly in the map. As spherical robots require stable rolling, we give a balance reward to prevent the spherical robot from shaking left and right

Penalty prevent robots from making inappropriate actions, including action rate penalty, torque penalty time penalty and overturn penalty. These penalties encourage stable motion and shorter path lengths.

Soft constraints encourage the spherical robot not to exceed safety limits [21], which is suitable for restricting objects that cannot be directly controlled. When the restriction is exceeded, a penalty will be imposed. Otherwise, no penalty or reward will be given. Soft constraints are applied to joint angles and velocities, as well as the linear and angular velocities of the base link.

TABLE II
REWARDS

Type	Term	Description	Weight
Positive reward	success	$\begin{cases}100, d \le d_s \wedge v_b \le v_s \\ 0 \quad , d > d_s \vee v_b > v_s\end{cases}$	20
	distance	$exp\left[-\left(\frac{p_c - p_b}{\sigma}\right)^2\right]$	1.5
	approaching target	$\begin{cases}1 \ , d^t - d^{t-1} < 0 \\ -1, d^t - d^{t-1} \ge 0\end{cases}$	0.1
	balance	$exp\left({w_{bx}}^2 + {w_{by}}^2\right)$	0.4
Penalty	torque	$\tau_{1t}^2 + \tau_{2t}^2$	-0.0001
	action rate	$(a_t - a_{t-1})^2$	-0.004
	time	1	-0.5
	overturn	1, if $q_r^2 > 1 \vee q_p^2 > 1$	-0.5
Soft constraint	base x velocity	1, if $v_{bx} > 1.5$	-1
	yaw angle velocity	1, if $\omega_{bz} > 0.72 v_{bx}$	-1

* d: distance to target; d_s: the maximum distance considered as target arrival; d_t: distance to target at time t; τ_{1t}, τ_{2t}: torques of Joint 1 and Joint 2 at time t; q_p: pitch angle.

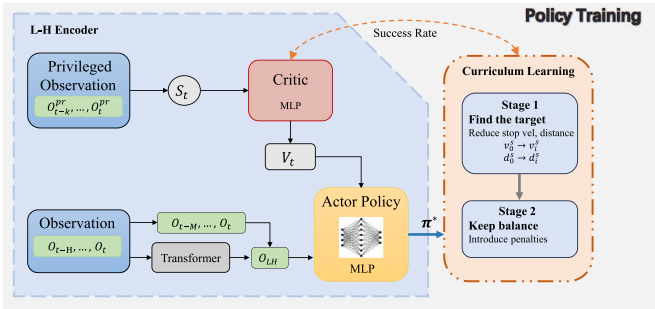

Fig. 2. Overview of policy learning framework.

Theoretically, the kinematics of a spherical robot are as follows:

$$\omega_{bz} = v_{bx} \tan q_r / R \tag{5}$$

where ω_{bz} is the yaw angular velocity of the robot during turning, v_{bx} is the forward linear velocity of the robot, q_r and R are the roll angle and radius of spherical shell. Therefore, the spherical robot has a minimum turning radius limit. We limit the size of the ω_{bz} instruction: $\omega_{bz} \le v_{bx}/1.386$.

E. Policy Learning

1) Long History Encoded Network: The entire control problem is formulated as a Markov Decision Process (MDP). We employ the PPO algorithm to solve for the optimal policy. The PPO algorithm is characterized by its structural simplicity and its mechanism for constraining the magnitude of policy updates. The structure of the algorithm is illustrated in Fig. 2.

PPO consists of two networks: a critic network and an actor network [21]. The critic network is used to estimate the value function of the MDP based on a predefined reward function. It

operates exclusively in simulator, where it receives privileged information o_t^{pr} as input to compute accurate value estimations. The critic network is an MLP network. We integrate the most recent three privileged observations $\{o_t^{pr}, o_{t-1}^{pr}, o_{t-2}^{pr}\}$ as the input to the network. The actor network is also an MLP network. We encode the previous H time steps observations $\{o_t, o_{t-1}, \ldots, o_{t-H+1}\}$ by transformer and combine it with the recent three observations $\{o_t, o_{t-1}, o_{t-2}\}$ as the input to the actor network.

2) Curriculum Learning: When providing all the reward functions to the robot, we find that it did not move toward the target as expected. The reason is that the task setup was too complex, as it required the ball to reach the target point quickly while also operating smoothly. The primary reward function $r_{success}$ is a sparse reward. The robot will be easily stuck in a local optimum. Therefore, we drew on the concept of curriculum learning [6], where the robot progresses to the next task after performing well on the current one. Each time a new curriculum is learned, training is conducted based on the previous policy.

Curriculum learning needs to address two issues: (a) How to evaluate the performance of a task; (b) What curriculum to set up during training. We evaluate the performance of a task with success rate $g_s(\pi_n)$. The task success flag is defined as $b_s(s_t)$:

$$b_s(s_t) = \begin{cases} 1 & d \le d_s \wedge v \le v_s \\ 0 & d > d_s \vee v > v_s \end{cases} \tag{6}$$

where d is the distance between spherical robot and target point, d_s is the stopping range, v is the robot absolute velocity in the ground coordinate system, v_s is the primary stopping criteria. We expect the robot to reach the target point and stop, so we limit the d and v. An episode is terminated and an environment is reset, if any of the following conditions are met: toppling, boundary violation, timeout, or successful task completion. After sufficient training, we execute the optimal policy π^* and take N reset environments. We use the successful portion to calculate the success rate $g_s(\pi_n)$:

$$g_s(\pi_n^*) = \sum_{1}^{N} b_s(s_t)/N \tag{7}$$

The curriculum consists of two stages. The first stage aims to make the robot move toward the target point, without incorporating penalty reward functions. In this stage, d_{min} and v_{min} are gradually narrowed down after $g_s(\pi_n^*) > 0.7$. In the second stage, we introduce action rate penalty, torque penalty and balance penalty for robot.

IV. Experiments

In this section, we present the results of simulation and real-world experiments. This work is the first instance of applying directly point-to-point control to a spherical robot, also the first to implement a sim2real transfer for this type of robot. The training and simulation are implemented on 2048 parallel environments in IsaacGym. We conducted 2000 training epochs, with the entire training process taking approximately 7.5 hours. Actions are output at 50 Hz. In real world experiments, our strategy is applied to a 2-dof spherical robot equipped with an Intel NUC for primary control. The experimental environments include flat ground, uneven gravel terrain, and an environment with disturbances.

TABLE III
Ablation Studies in the Simulation

Time [s]	Approach	SR [%]	SPL [%]	CLS [m]	Balance Metric[%]
40	w/o $r_{approach}$	60.17	32.80	0.8370	71.06±3.34
	w/o History	54.49	29.82	1.1740	58.32±3.32
	w/o LH Encoder	65.82	40.81	0.7275	68.71±4.65
	w/o CL	39.04	21.74	1.5770	73.54±2.70
	CNN Encoder	73.25	50.47	0.5164	74.13±3.54
	LSTM Encoder	73.84	53.32	0.5336	75.14±3.32
	proposed	**73.88**	**54.79**	**0.5238**	**74.92±3.01**
60	w/o $r_{approach}$	70.31	37.58	0.6032	74.60±3.27
	w/o History	68.36	34.41	0.8102	60.87±3.22
	w/o LH Encoder	77.15	49.57	0.3176	70.18±2.73
	w/o CL	45.70	29.18	1.3286	76.44±2.67
	CNN Encoder	88.35	60.83	0.1998	75.38±2.89
	LSTM Encoder	88.94	62.79	0.2105	74.24±2.42
	proposed	**88.87**	**63.75**	**0.2092**	**75.52±2.32**

Metrics: We evaluate our approach using three key metrics. Success Rate (**SR**) is the primary metric for overall task completion. which states the success rate of reaching the target point in the given time. To assess path efficiency, we employ the Success weighted by Path Length (**SP**L), where a higher value indicates a more optimal trajectory. Closest Distance to Goal (**CLS**) reflects the robot's ability to approach the target. Finally, to quantify motion stability, we introduce a **Balance Metric**, which is calculated as the average of balance reward over an entire trajectory.

A. Simulation Experiments

In simulation, we designed and executed a comprehensive set of ablation studies to evaluate the contribution of each core component within our proposed end-to-end control framework. We design the following four ablation experiments:

- **w/o $\mathbf{r_{approach}}$** : Policy trained without approaching target reward.
- **w/o History**: Policy trained based on the latest observation, without history information.
- **w/o LH Encode**r: Policy trained based on short history observation, without long history encoder.
- **CNN Encoder**: Replace Transformer encoder with CNN.
- **LSTM Encoder**: Replace Transformer encoder with LSTM.

As shown in Fig. 3, compared to the w/o CL and w/o LH Encoder baselines, our method generates trajectories that are shorter and smoother, reaching the target faster. Without curriculum learning, stable roll performance is achieved, but there is an increase in time-to-target and path length. In the absence of the LH Encoder, using short-term historical data, both the roll angle stability and the path efficiency are reduced.

As shown in Table III, the proposed approach achieves superior performance across key metrics: SR, SPL and CLS. Our method also achieves good performance in terms of balance. As the simulation time increases, the success rate also increases and our SR can reach 88.87% at 60 s. This shows that although the robot sometimes goes wrong, it will correct the path and move to the target point.

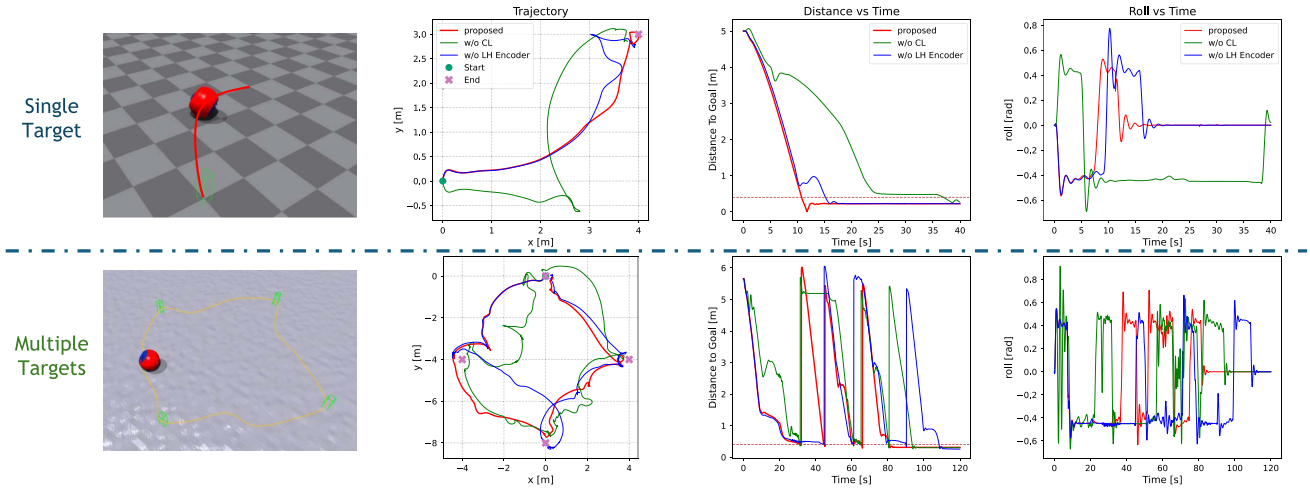

Fig. 3. Experiments in the simulation. The upper illustrates a spherical robot's single-target-point navigation example, covering the trajectory plot, distance-to-target plot, and roll angle variation plot, with the red color indicating the proposed method. The lower figure shows the robot's multi-target-point tracking example.

The w/o CL variant performs the worst across all key metrics, but performs well in terms of balance. This shows that for a complex, end-to-end robotic control task, training on the full task difficulty from the outset is inefficient and prone to convergence to suboptimal policies. The w/o $r_{approach}$ variant achieved inferior results across all metrics. Although it could maintain physical stability, the spherical robot may explore randomly or wander aimlessly due to the absence of a clear approaching signal. This reward provides a dense and effective gradient that is essential for guiding the agent's learning process toward the goal.

The w/o History variant demonstrated extremely poor performance in terms of SR and SPL. This indicates that relying on the current observation is insufficient for the agent to infer its own dynamic state, leading to short-sighted and unstable control policies. Furthermore, while the w/o LH Encoder variant outperformed the history-less version, it still lags behind ours. The long-history encoding mechanism enables the agent to capture higher-order dynamics, facilitating more predictive and smoother control. This improvement is directly reflected in its higher SPL and CLS.

In the network architecture section, compared with the CNN Encoder and LSTM Encoder, the Transformer network architecture demonstrates a distinct advantage in terms of the SPL metric, as the Transformer Encoder is capable of optimizing shorter trajectories. The impact of the encoding time step Hon the encoder can be observed in Fig. 4. As the time step increases, the growth rates of SR and SPL gradually slow down; especially when H exceeds 10, the improvement in metrics becomes limited. Considering that a larger H will increase the network size and reduce training efficiency, and in physical deployment scenarios, excessive historical information will lead to error accumulation at each time step, which may instead decrease the success rate, we therefore set H=10.

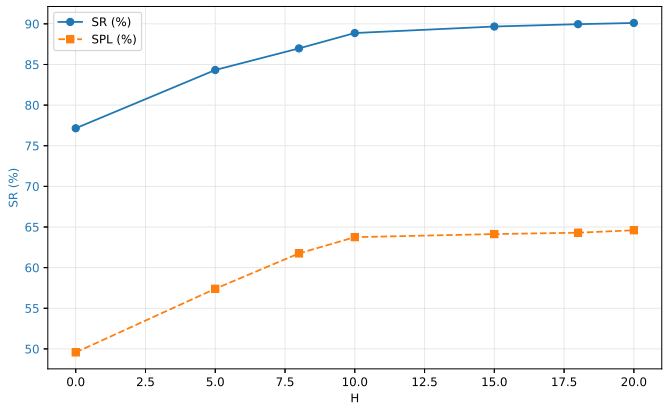

Fig. 4. The influence curves of H size in LH-Encoder on SR and SPL.

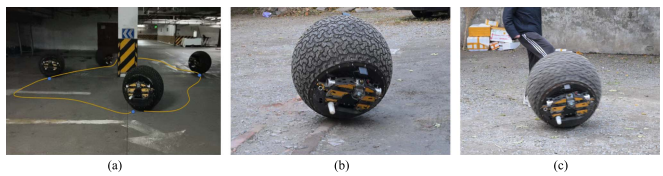

Fig. 5. Spherical robot experimental environment. (a) plane, (b) complex terrain, (c) complex terrain with disturbances.

B. Real-World Experiments

In this section, we conducted experiments on a real spherical robot. To verify the advantages of our sim2real method, we designed a comparative experiment with domain random (**DR**). To verify our advantages over traditional methods, we designed a comparative experiment with **MPC**. In the MPC method, we first use MPC to plan the forward speed v_{bx} and the roll angle q_r. Then we use PID to control v_{bx} and q_r. The real experiment environments include flat ground, uneven gravel roads and scenarios with disturbances, as in Fig. 5. In the scenarios with disturbances, the spherical robot is kicked at random times. We test each method and each environment 40 times with random targets, each test lasting 1 minute.

We adopted the MC-CMA-ES method to identify 37 parameters of spherical robot in the simulator, including link parameters, motor parameters, and friction parameters. As shown

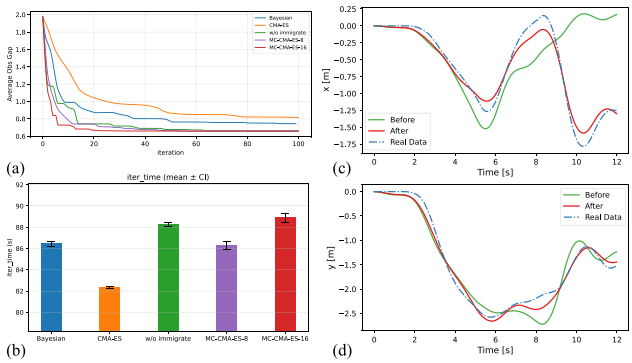

Fig. 6. Result of system identification. (Left: Comparison of different identification methods; Right: Results of parameter identification on x and y coordinates). (a) Curve of sim2real gap per average frame with same population, (b) average calculation time per round, (c) tracking curve in the x-direction, (d) tracking curve in the y-direction.

TABLE IV
REAL-WORLD EXPERIMENT RESULTS UNDER VARIOUS ENVIRONMENTS

Time [s]	Approach	SR [%]	SPL [%]	Balance Metric[%]
Plane	DR	75	37.23	71.75±3.64
	MPC	85	62.15	56.22±6.03
	SysID	**87.5**	**58.82**	**70.09±5.28**
Multiple Targets	DR	57.5	31.32	71.95±4.22
	MPC	65	36.07	52.65±7.74
	SysID	**80**	**46.10**	**70.54±5.68**
Complex Terrain	DR	55	20.32	54.93±5.89
	MPC	52.5	21.49	41.18±9.09
	SysID	**70**	**33.19**	**52.03±7.42**
Disturbances	DR	35	8.75	45.07±7.03
	MPC	20	6.33	36.12±9.22
	SysID	**52.5**	**21.86**	**41.55±8.51**

in Fig. 6(a), compared with the basic CMA-ES and Bayesian methods, the multi-population CMA-ES scheme can expand the search range and avoid falling into local optima. Moreover, the introduction of the migration mechanism improves the search convergence speed. As indicated in Fig. 6(b), the MC-CMA-ES method does not lead to a significant increase in computational resource consumption when the number of individuals is kept the same. The positional error continuously increases without SYSID, as shown in Fig. 6(c) and (d), which negatively impacts the navigation of the spherical robot.

As shown in Table IV and Fig. 7, our RL approach demonstrates a significant advantage in terms of balance and trajectory compared to MPC. Although MPC exhibits robust performance in the SR and SPL, the roll angle changes dramatically during the test. This shortcoming is more obvious when conducting experiments with multiple target points. The advantages of reinforcement learning methods are more pronounced in complex terrains and disturbed environments. Because the dynamic model underlying MPC lacks precise modeling of terrain and external forces, rendering it prone to failure in unknown environments, especially when subjected to disturbances. In contrast, reinforcement learning can learn a strategy to handle complex and diverse environments during simulation, thus achieving

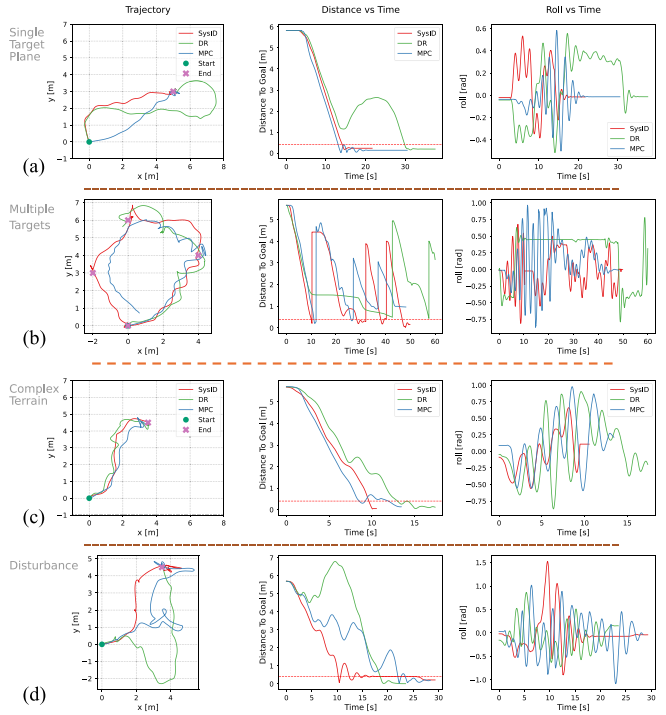

Fig. 7. Result of real world experiments. (Left: trajectories under different conditions and methods; Middle: distance to the target point over time; Right: Roll angle change over time). (a) Flat ground single target experimental environment, (b) flat ground multiple targets experimental environment, (c) gravel roads single target experimental environment, (d) gravel roads single target experimental environment with disturbances.

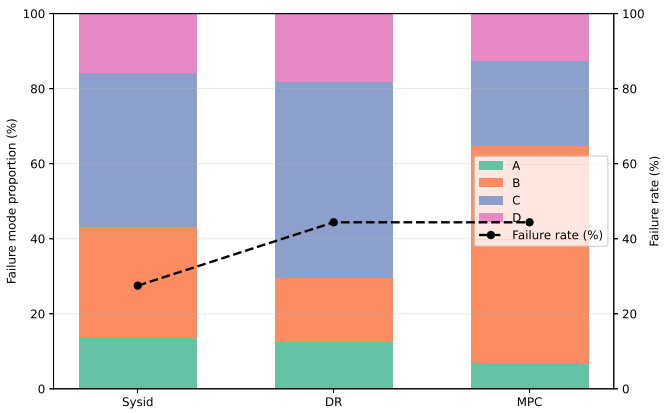

Fig. 8. Distribution Map of Failure Modes. (Left: Sysid; Middle: Domain Random; Right: MPC) Types of failure: **A**: Stuck and unable to move; **B**: Excessively large shaking amplitude, causing dangers; **C**: Failure to reach the target within the time limit; **D**: Driving out of the specified range.

superior robustness and stability. Fig. 8 shows the distribution of failure modes across different methods. It can be observed that the primary failure mode of MPC is excessive shaking, while Sysid and DR mainly fail due to timeout.

In Sim2Real transfer experiments, our system identification method outperforms DR across all key metrics. While Domain Randomization learns a robust policy capable of functioning across a wide range of simulated environments, this policy is inherently conservative and suboptimal. The DR-trained policy often exhibits hesitation, overcompensation, or a preference for safer, redundant paths in the real world.

V. Conclusion

In this work, we propose a RL based controller for the point-to-point control of a spherical robot. We design a framework for sim2real transfer and policy training based on the characteristics of the spherical robot and the point-to-point task. Our method demonstrates high efficiency and stability in experiments. Compared to traditional methods, our approach yields smoother robot trajectories and exhibits better adaptability to uncertain environ- ments. It achieves a high success rate in both single-goal navigation and navigation to multiple sequential targets. In the future, we expect that this reinforcement learning-based point tracking method can be applied to a broader range of robot platforms. We also aim to enable the spherical robot to navigate in more complex environments by incorporating more sensory information.

References

[1] Y. Liu et al., "Direction and trajectory tracking control for nonholonomic spherical robot by combining sliding mode controller and model prediction controller," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 11617–11624, Oct. 2022.

[2] R. Chase and A. Pandya, "A review of active mechanical driving principles of spherical robots," *Robotics*, vol. 1, no. 1, pp. 3–23, 2012.

[3] Q. Zhan, T. Zhou, M. Chen, and S. Cai, "Dynamic trajectory planning of a spherical mobile robot," in *Proc. 2006 IEEE Conf. Robot., Automat. Mechatron.*, 2006, pp. 1–6.

[4] S. Zihao, W. Bin, and Z. Ting, "Trajectory tracking control of a spherical robot based on adaptive PID algorithm," in *Proc. 2019 Chin. Control Decis. Conf.*, 2019, pp. 5171–5175.

[5] C. Zhang, D. Chu, S. Liu, Z. Deng, C. Wu, and X. Su, "Trajectory planning and tracking for autonomous vehicle based on state lattice and model predictive control," *IEEE Intell. Transp. Syst. Mag.*, vol. 11, no. 2, pp. 29–40, Summer 2019.

[6] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Sci. Robot.*, vol. 7, no. 62, 2022, Art. no. eabk2822. Accessed: Feb. 05, 2024. [Online]. Available: https://www.science.org/doi/10.1126/scirobotics.abk2822

[7] L. Smith, I. Kostrikov, and S. Levine, "A walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning," Aug. 16, 2022. Accessed: Feb. 6, 2024, *arXiv:2208.07860*.

[8] X. Gu et al., "Advancing humanoid locomotion: Mastering challenging terrains with denoising world model learning," in *Proc. Robot.: Sci. Syst. XX, Robot.: Sci. Syst. Found.*, Jul. 2024, doi: 10.15607/RSS.2024.XX.058.

[9] H. Liang, L. Cong, N. Hendrich, S. Li, F. Sun, and J. Zhang, "Multifingered grasping based on multimodal reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 1174–1181, Apr. 2022.

[10] V. Pong, S. Gu, M. Dalal, and S. Levine, "Temporal difference models: Model-free deep rl for model-based control," 2018, *arXiv:1802.09081*.

[11] H. Ping, P. Fu, X. Guan, Y. Wang, and G. Li, "Trajectory planning for spherical robot based on differential flatness," in *Proc. 2024 China Automat. Congr.*, Qingdao, China, 2024, pp. 1196–1201.

[12] Y. Liu et al., "Adaptive MPC-based multi-terrain trajectory tracking framework for mobile spherical robots," *IEEE/ASME Trans. Mechatron.*, vol. 30, no. 6, pp. 6785–6797, Dec. 2025.

[13] T. Salzmann, E. Kaufmann, J. Arrizabalaga, M. Pavone, D. Scaramuzza, and M. Ryll, "Real-time neural MPC: Deep learning model predictive control for quadrotors and agile robotic platforms," *IEEE Robot. Autom. Lett.*, vol. 8, no. 4, pp. 2397–2404, Apr. 2023.

[14] U. Rosolia and F. Borrelli, "Learning model predictive control for iterative tasks. A data-driven control framework," *IEEE Trans. Autom. Control*, vol. 63, no. 7, pp. 1883–1896, Jul. 2018.

[15] S. Moazami, H. Zargarzadeh, and S. Palanki, "Kinematics of spherical robots rolling over 3D terrains," *Complexity*, vol. 2019, 2019, Art. no. 7543969.

[16] V. Dwaracherla, S. Thakar, L. Vachhani, A. Gupta, A. Yadav, and S. Modi, "Motion planning for point-to-point navigation of spherical robot using position feedback," *IEEE/ASME Trans. Mechatron.*, vol. 24, no. 5, pp. 2416–2426, Oct. 2019.

[17] X. Cheng, K. Shi, A. Agarwal, and D. Pathak, "Extreme parkour with legged robots," in *Proc. 2024 IEEE Int. Conf. Robot. Automat.*, 2024, pp. 11443–11450.

[18] S. Gangapurwala, M. Geisert, R. Orsolino, M. Fallon, and I. Havoutis, "RLOC: Terrain-aware legged locomotion using reinforcement learning and optimal control," *IEEE Trans. Robot.*, vol. 38, no. 5, pp. 2908–2927, Oct. 2022.

[19] X. Gu, Y.-J. Wang, and J. Chen, "Humanoid-gym: Reinforcement learning for humanoid robot with zero-shot sim2real transfer," 2024, *arXiv:2404.05695*.

[20] Z. Li et al., "Reinforcement learning for robust parameterized locomotion control of bipedal robots," in *Proc. 2021 IEEE Int. Conf. Robot. Automat.*, 2021, pp. 2811–2817.

[21] I. Dadiotis, M. Mittal, N. Tsagarakis, and M. Hutter, "Dynamic object goal pushing with mobile manipulators through model-free constrained reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, 2025, pp. 13363–13369.

[22] I. Ozdamar, D. Sirintuna, R. Arbaud, and A. Ajoudani, "Pushing in the dark: A reactive pushing strategy for mobile robots using tactile feedback," *IEEE Robot. Autom. Lett.*, vol. 9, no. 8, pp. 6824–6831, Aug. 2024.

[23] T. Chaffre et al., "Sim-to-real transfer of adaptive control parameters for AUV stabilisation under current disturbance," *Int. J. Robot. Res.*, vol. 44, no. 3, pp. 407–430, Mar. 2025.

[24] M. Memmel, A. Wagenmaker, C. Zhu, P. Yin, D. Fox, and A. Gupta, "ASID: Active exploration for system identification in robotic manipulation," Jun. 27, 2024, *arXiv:2404.12308*.

[25] Y.-Y. Tsai, H. Xu, Z. Ding, C. Zhang, E. Johns, and B. Huang, "DROID: Minimizing the reality gap using single-shot human demonstration," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 3168–3175, Apr. 2021.

[26] F. Ramos, R. C. Possas, and D. Fox, "BayesSim: Adaptive domain randomization via probabilistic inference for robotics simulators," Jun. 04, 2019, *arXiv:1906.01728*.

[27] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, "ConRFT: A reinforced fine-tuning method for VLA models via consistency policy," Apr. 14, 2025, *arXiv:2502.05450*.

[28] P. Yin et al., "Rapidly adapting policies to the real world via simulation-guided fine-tuning," Feb. 04, 2025,*arXiv:2502.02705*.

[29] L. Smith, J. C. Kew, X. B. Peng, S. Ha, J. Tan, and S. Levine, "Legged robots that keep on learning: Fine-tuning locomotion policies in the real world," in *Proc. Int. Conf. Robot. Automat.*, 2022, pp. 1593–1599.

[30] Q. Zhan, Y. Cai, and C. Yan, "Design, analysis and experiments of an omni-directional spherical robot," in *Proc. 2011 IEEE Int. Conf. Robot. Automat.*, 2011, pp. 4921–4926.

[31] Y. Cai, Q. Zhan, X. Xi, and A. Rahmani, "Inverse kinematics identification of a spherical robot based on BP neural networks," in *Proc. 6th IEEE Conf. Ind. Electron. Appl.*, 2011, pp. 2114–2119.

[32] A. Auger and N. Hansen, "A restart CMA evolution strategy with increasing population size," in *Proc. 2005 IEEE Congr. Evol. Computation*, 2005, vol. 2, pp. 1769–1776.

[33] D. Redon et al., "Massively parallel CMA_ES with increasing population," *Concurrency Computation: Pract. Experience*, vol. 37, no. 27-28, 2025, Art. no. e70362.

[34] A. Ahrari, K. Deb, and M. Preuss, "Multimodal optimization by covariance matrix self-adaptation evolution strategy with repelling subpopulations," *Evol. Computation*, vol. 25, no. 3, pp. 439–471, 2017.